

1.INTRODUCTION

In today's digital era, programming has become an essential skill for students, professionals, and software developers. Traditional programming environments require users to install compilers, interpreters, and various development tools on their local systems before they can write and execute programs. This process often leads to compatibility issues, dependency conflicts, platform-specific limitations, and increased setup complexity, especially for beginners. These challenges create barriers to learning and reduce the accessibility of programming resources.

To overcome these limitations, **DYC CODING CAMPUS - A WEB-BASED ONLINE COMPILER PLATFORM** s have emerged as an effective solution. An online compiler enables users to write, compile, and execute code directly through a web browser without requiring any software installation. Such platforms provide a convenient and accessible environment for coding practice, learning, and testing programs from any device with internet connectivity.

The proposed system, "Web-Based Online Compiler and Code Execution Platform," is designed to provide a secure, scalable, and user-friendly coding environment. The platform supports multiple programming languages including Python, JavaScript, C, C++, and Java. It utilizes Docker containerization technology to execute user programs in isolated environments, ensuring security and preventing unauthorized access to server resources. The system also includes features such as user authentication, code management, execution history tracking, and administrative controls.

The application is developed using React.js for the frontend, FastAPI for backend services, MongoDB for data storage, and Docker for secure code execution. By integrating modern web technologies with containerized execution environments, the proposed platform offers a reliable and efficient solution for online programming education, skill development, and coding assessment.

2.SYSTEM REQUIREMENTS

HARDWARE REQUIREMENTS

The following hardware components are required for the development and deployment of the proposed system:

Component	Specification
Processor	Intel Core i5 / AMD Ryzen 5 or Higher
RAM	Minimum 8 GB
Storage	Minimum 100 GB Free Disk Space
Network	Broadband Internet Connection
Display	1366 × 768 Resolution or Higher
Input Devices	Keyboard and Mouse
Server Configuration	Multi-Core Processor, 8 GB RAM or Above

SOFTWARE REQUIREMENTS

The proposed system utilizes modern web technologies and development tools for efficient implementation.

Software Component	Specification
Operating System	Windows 10/11, Linux, macOS
Frontend Framework	React.js
Backend Framework	FastAPI (Python)
Programming Language	Python, JavaScript
Database	MongoDB
Containerization Platform	Docker
Code Editor	Monaco Editor
Authentication	JSON Web Token (JWT)
API Testing Tool	Postman
Version Control System	Git
Repository Management	GitHub
Web Browser	Google Chrome, Microsoft Edge, Mozilla Firefox
Development Environment	Visual Studio Code

3.PROBLEM STATEMENT

The existing programming environment faces several challenges:

- Installation of compilers and interpreters is required for different programming languages.
- Software compatibility issues occur across different operating systems.
- Dependency conflicts arise when multiple programming environments are installed on the same system.
- Beginners find it difficult to configure and set up development environments.
- Limited accessibility as coding can only be performed on systems with the required software installed.
- Managing programming environments in educational institutions is time-consuming.
- Executing user code directly on servers can create serious security risks.
- Existing solutions may not provide proper isolation between users' programs.
- Lack of centralized code storage and program management features.
- Difficulty in tracking execution history and user activities.
- High maintenance effort is required to support multiple programming languages.
- Need for a secure, scalable, and browser-based platform for code execution.

Therefore, a secure web-based online compiler is required to provide multi-language support, isolated code execution using Docker containers, centralized program management, and easy accessibility through a web browser.

4.EXISTING SYSTEM ANALYSIS

1. TRADITIONAL OFFLINE COMPILERS AND IDES

FEATURES

- Local code development and execution.
- Supports multiple programming languages.
- Advanced debugging tools.
- Offline accessibility.
- High performance execution.

DRAWBACKS

- Requires software installation and configuration.
 - Dependency and compatibility issues.
 - Platform-specific setup requirements.
 - Difficult for beginners to configure.
 - Consumes local system resources.
-

2. REPLIT

FEATURES

- Browser-based code editor.
- Multi-language support.
- Cloud-based project storage.
- Real-time collaboration.
- Instant code execution.

DRAWBACKS

- Limited features in the free version.
 - Performance depends on internet connectivity.
 - Resource restrictions for free users.
 - Advanced features require paid subscriptions.
-

3. HACKERRANK

FEATURES

- Online code execution.
- Programming challenges and assessments.
- Multi-language support.
- Automated evaluation system.
- Performance analytics.

DRAWBACKS

- Primarily designed for coding assessments.
 - Limited project management features.
 - Restricted customization options.
 - Not suitable for general-purpose development.
-

4. CODECHEF IDE

FEATURES

- Browser-based compiler.
- Supports multiple programming languages.
- Fast code execution.
- User-friendly interface.

DRAWBACKS

- Limited file management capabilities.
 - No advanced project development features.
 - Minimal collaboration support.
 - Limited personalization options.
-

5. JDOODLE

FEATURES

- Online compiler and code execution.
- Supports multiple languages.
- Simple and lightweight interface.
- Quick code testing environment.

DRAWBACKS

- Limited execution resources.
 - Restricted storage capabilities.
 - Advanced features require premium plans.
 - Limited support for large-scale projects.
-

5.PROPOSED SYSTEM

The proposed system is a **DYC CODING CAMPUS - A WEB-BASED ONLINE COMPILER PLATFORM** that enables users to write, compile, execute, and manage programs directly through a web browser without installing any programming software on their local machines. The system utilizes Docker containerization technology to provide secure and isolated code execution.

FEATURES

- Browser-based code editor with syntax highlighting and auto-indentation.
- Multi-language support such as Python, Java, JavaScript, C, and C++.
- Secure code execution using Docker containers.
- User registration, authentication, and profile management.
- Program creation, editing, saving, downloading, and deletion.
- Real-time output and error display.
- Execution history tracking and management.
- AI-powered code suggestions, debugging assistance, and code optimization support.
- Online coding assessments with automatic evaluation and scoring.
- Coding challenges for skill development and problem-solving practice.
- Leaderboard system to rank users based on challenge and assessment performance.
- Certificate generation for successful completion of assessments and coding activities.
- Code sharing functionality for collaboration and learning.
- Administrative dashboard for user management and activity monitoring.
- Analytics and performance tracking for users and administrators.

ADVANTAGES

- No software installation required.
- Accessible from any device with internet connectivity.
- Eliminates dependency and compatibility issues.
- Provides secure execution through isolated Docker containers.
- Supports multiple programming languages in a single platform.
- Enhances learning through AI- Assisted coding support.
- Encourages competitive learning through challenges and leaderboards.
- Simplifies coding assessments and result management.
- Centralized storage and management of programs.
- Scalable and extensible architecture for future enhancements.

6.OBJECTIVES OF THE SYSTEM

The main objectives of the proposed Web-Based Online Compiler and Code Execution Platform are:

- To provide a browser-based programming environment without requiring software installation.
- To support multiple programming languages such as Python, Java, JavaScript, C, and C++.
- To enable secure code execution using Docker containerization technology.
- To provide an interactive code editor with syntax highlighting and user-friendly features.
- To allow users to create, save, edit, download, and manage source code files.
- To display program output and error messages in real time.
- To maintain execution history for future reference and analysis.
- To provide AI- Assisted coding support for improved learning and development.
- To conduct online coding assessments and automated evaluations.
- To offer coding challenges that enhance programming and problem-solving skills.
- To implement a leaderboard system that encourages healthy competition among users.
- To generate certificates for successful completion of coding activities and assessments.
- To provide administrative tools for user management and activity monitoring.
- To develop a scalable and secure platform suitable for educational and professional use.

7.SYSTEM ARCHITECTURE

The proposed Web-Based Online Compiler and Code Execution Platform follows a client-server architecture that integrates a React.js frontend, FastAPI backend, MongoDB database, and Docker-based execution environment. The architecture is designed to provide secure, scalable, and efficient code execution while ensuring a seamless user experience.

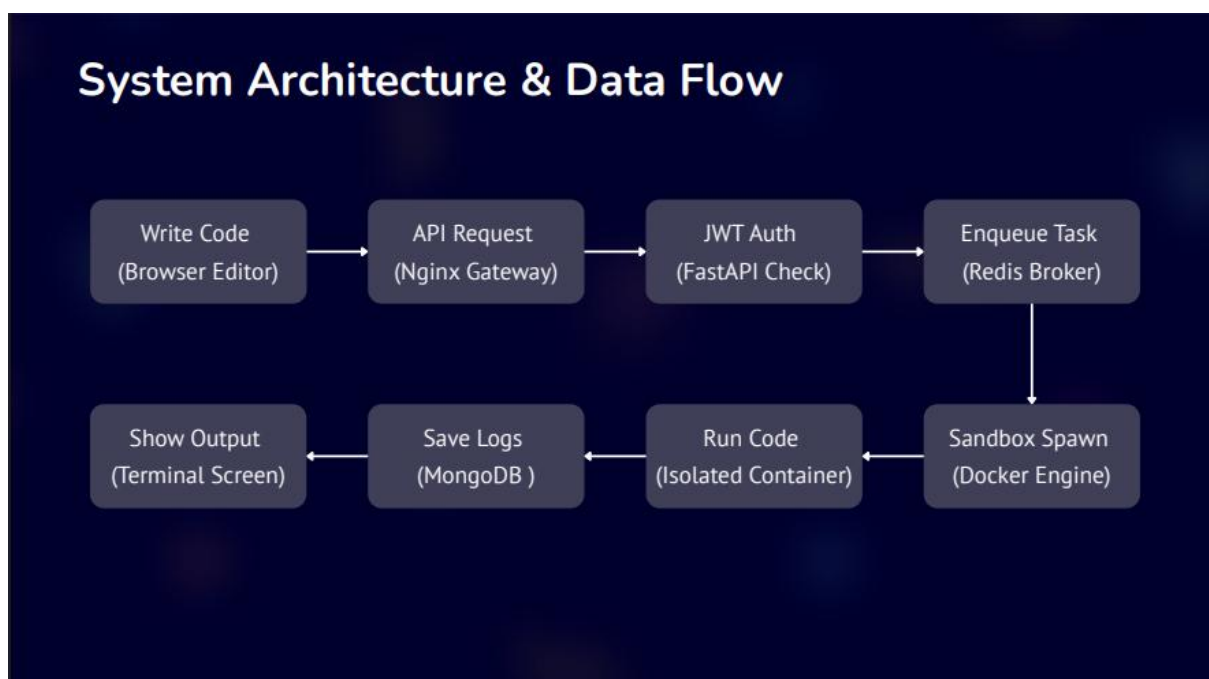
The user interacts with the system through a web browser, where the React.js frontend provides features such as code editing, file management, coding challenges, assessments, and user account management. User requests are sent to the FastAPI backend through RESTful APIs.

The FastAPI backend is responsible for handling authentication, program management, execution requests, assessment processing, leaderboard management, and communication with the database. All user information, source code files, execution history, challenge data, assessment records, and leaderboard information are stored in MongoDB.

When a user executes a program, the backend forwards the code to the Docker Execution Engine. The execution engine creates an isolated Docker container based on the selected programming language, compiles and executes the code, captures the output or errors, and returns the results to the backend. After execution, the container is automatically removed to ensure security and efficient resource utilization.

This architecture provides platform independence, secure code execution, centralized data management, and support for future enhancements such as AI-powered coding assistance, collaborative coding, and advanced analytics.

SYSTEM FLOW:



8.DESCRPTION OF THE MODULES

AUTHENTICATION

- **Secure Registration:** Registers users with input validation and duplicate account detection.
- **Password Encryption:** Encrypts user passwords using bcrypt before storing them in the database.
- **Stateless Session Control:** Utilizes JSON Web Tokens (JWT) for secure user authentication and session management.
- **Role-Based Access Control (RBAC):** Restricts access to system resources and administrative functionalities based on user roles.
- **Security Middleware:** Validates JWT tokens for every protected API request to prevent unauthorized access.

SANDBOXED CODE COMPILATION ENGINE

- **Dynamic Docker Containers:** Creates isolated Docker containers for each code execution request.
- **Multi-Language Support:** Supports Python, Java, JavaScript, C, and C++ programming languages.
- **Strict Resource Constraints:** Applies memory, CPU, and process limits to ensure controlled execution.
- **Host System Protection:** Executes code within isolated environments to safeguard the host system.
- **Ephemeral Execution Environment:** Automatically removes temporary execution environments after program completion.
- **Network Isolation:** Prevents external network access during code execution to enhance security.

CODE SNIPPET SHARING

- **Instant Link Generation:** Generates unique shareable links for code snippets.
- **Privacy Controls:** Supports public, private, and time-limited sharing options.

- **Fork and Clone Support:** Allows users to copy and modify shared code within their own workspace.
- **Syntax Highlighted Viewer:** Displays shared code in a read-only editor with proper formatting.
- **Activity Tracking:** Records code views and forks for engagement monitoring.

ONLINE ASSESSMENTS & CERTIFICATION

- **Proctored Online Assessments:** Conducts secure time-based programming assessments.
- **Anti-Cheat Monitoring:** Detects suspicious activities and enforces assessment integrity.
- **Automated Evaluation:** Evaluates submissions using predefined test cases and scoring criteria.
- **Dynamic Certificate Generation:** Generates digital certificates for successful assessment completion.
- **Certificate Verification:** Provides unique verification mechanisms to validate certificate authenticity.
- **Public Verification Portal:** Enables third parties to verify issued certificates online.

CODING CHALLENGES & LEADERBOARD

- **Categorized Challenge Repository:** Organizes challenges by difficulty levels and programming topics.
- **Automated Test Case Execution:** Validates solutions against multiple test cases.
- **Result Analysis:** Displays detailed execution outcomes such as Passed, Failed, Runtime Error, and Time Limit Exceeded.
- **Dynamic Scoring System:** Awards points based on challenge difficulty and performance.
- **Real-Time Leaderboard:** Maintains rankings based on user achievements and accumulated scores.
- **Performance Tracking:** Monitors user progress across coding challenges.

GEMINI-POWERED SMART ASSISTANT

- **Code Explanation:** Provides human-readable explanations for source code and algorithms.
- **Debugging Assistance:** Identifies errors and suggests possible corrections.
- **Intelligent Code Suggestions:** Offers context- Aware code completion and recommendations.
- **Code Optimization Support:** Recommends improvements for efficiency and readability.
- **Usage Control:** Implements request limits to ensure fair resource utilization.
- **Response Caching:** Reduces redundant AI requests through optimized data storage mechanisms.

ADMINISTRATIVE CONTROL CENTER

- **User Management:** Enables administrators to monitor, block, unblock, and manage user accounts.
- **Challenge Management:** Supports creation and maintenance of coding challenges and test cases.
- **Assessment Management:** Allows administrators to create assessments and configure evaluation criteria.
- **Certificate Audit System:** Tracks issued certificates and verification records.
- **Activity Monitoring:** Maintains logs of user actions and platform activities.
- **Analytics Dashboard:** Displays key metrics such as code executions, active users, language usage, and assessment statistics.

9.DATABASE DESIGN

The proposed system utilizes MongoDB as its primary database management system. The database is designed to support scalability, flexibility, and efficient data management. The complete system consists of approximately 21 collections that manage various platform functionalities including authentication, code execution, assessments, certificates, leaderboards, AI interactions, activity logs, and administrative operations.

Among these collections, the following five serve as the core collections of the platform:

USERS COLLECTION

Stores user account information and authentication details.

Fields:

- Name
- Email
- Encrypted Password
- Role (Admin / Student)
- Account Creation Date

PROGRAMS COLLECTION

Maintains user-created programs and workspace files.

Fields:

- Program Title
- Programming Language
- Source Code
- User ID
- Last Modified Date

CODING_CHALLENGES COLLECTION

Contains coding problems and challenge-related information.

Fields:

- Challenge Title
- Difficulty Level
- Problem Statement
- Public Test Cases
- Hidden Test Cases
- Challenge Score

CERTIFICATES COLLECTION

Stores information related to generated digital certificates.

Fields:

- Student ID
- Assessment Score
- Issue Date

- Verification Hash
- Certificate ID

EXECUTION_HISTORY COLLECTION

Maintains records of all code execution activities.

Fields:

- Programming Language
- Source Code
- Console Output
- Execution Status
- Execution Timestamp

These core collections collectively support the primary functionalities of the platform, including user management, program storage, coding challenges, certificate generation, and secure code execution tracking.

10.CONCLUSION

The development and implementation of the Secure Web-Based Online Compiler and Code Execution Platform successfully address the critical challenges of resource management, security, and scalability in online programming environments.

By leveraging OS-level virtualization (Docker container sandboxing), the platform establishes a bulletproof security barrier that prevents untrusted user code from accessing the host filesystem, scanning local networks, or performing Remote Code Execution (RCE) attacks. Strict hardware caps on CPU cores, RAM consumption, and execution timeouts ensure complete defense against fork bombs and Denial of Service (DoS) attempts.

Furthermore, the integration of a decoupled architecture using a Redis Task Queue ensures high concurrency without server degradation, as execution workloads are efficiently buffered and consumed by dedicated background workers.

For academic integrity and administrative oversight, the platform incorporates:

- Real-time Page Visibility Monitoring (Anti-Cheat) to prevent exam tab-switching.
- Cryptographically Signed Certificates (SHA-256) for tamper-proof credentials.
- System Backup and Recovery Mechanisms to guarantee data resilience.

In conclusion, this platform provides an optimized, secure, and user-friendly ecosystem for universities, coding camps, and recruitment teams to conduct technical training and assessments with confidence.

11.FUTURE ENHANCEMENTS

The proposed system is designed with scalability and extensibility in mind, allowing several advanced features to be integrated in future versions.

- AI-powered personalized learning and coding recommendations.
- Integration with GitHub, GitLab, and other version control platforms.
- Mobile application support for Android and iOS devices.
- Multi-file project creation and execution support.
- Integration of additional programming languages and frameworks.
- Detailed performance analytics and personalized progress tracking.
- Offline code synchronization and cloud backup facilities.
- Enterprise-level assessment and recruitment management features.